

問 5

以下は、社員テーブル (employee) から社員名 (氏+名を連結したもの)、役職クラス (部長、課長は管理職、主任、担当は総合職、アシスタントは一般職とします) を出力するためのSQL命令ですが、2つ誤りがあります。誤りを指摘してください。

```
SELECT
  CONCAT(l_name, f_name),
  CASE
    WHERE class IN ('部長', '課長') THEN '管理職'
    WHERE class IN ('主任', '担当') THEN '総合職'
    ELSE THEN '一般職'
  END AS 役職クラス
FROM
  employee
;
```

chapter

2

より複雑な問い合わせを理解しよう

Chapter1 では、データ操作命令を用いることで、基本的なデータの検索について学習しました。しかし、イントロダクションでも述べたように、リレーショナルデータベースはテーブルと呼ばれる 2 次元の表を「リレーションシップ」によって互いに関連付け、複雑な構造データをも表現できるのが特長です。そこで本章では、互いに関連付けられたテーブルを結合したり、他のテーブルで得られた結果に基づいて異なるテーブルを検索するような、より「RDB 的な」問い合わせについて学習してみることにしましょう。

2-1 内部結合で複数テーブルのデータを結合したい INNER JOIN...ON 句

MySQL PostgreSQL SQLite SQL Server Oracle



RDBの世界では、更新時のデータの整合性を維持するために「正規化」という作業を行い、データを適切な単位に分割するのが一般的です (P.13)。では、分割されたデータは、それぞれテーブル単位に参照しなければならないのでしょうか。だとしたら、大変面倒なことです。SQLでは分割したテーブルを1つに「結合」するための機能を提供しています。結合を利用することで、複数のテーブルに対して1つのSQL命令でアクセスすることができます。

Q 社員テーブル (employee) と所属部署テーブル (depart) から氏名と所属部署名、役職を取り出してみましょう。ただし、退職済みの社員は除くものとし、所属部署コード、社員コードについて昇順で出力するものとします。

A 複数のテーブルを結合するには、INNER JOIN...ON 句を使用します。

```
> SELECT
>   e.l_name,
>   e.f_name,
>   d.depart_name,
>   e.class
> FROM
>   employee AS e
> INNER JOIN
>   depart AS d
> ON
>   e.depart_id=d.depart_id
> WHERE
>   e.retired <> 1
> ORDER BY
>   e.depart_id ASC,
>   e.s_id ASC
> ;
```

```
+-----+-----+-----+-----+
| l_name | f_name | depart_name | class |
+-----+-----+-----+-----+
```

```
+-----+-----+-----+-----+
| 藤井   | 雄太   | 第一営業部 | 主任   |
| 藤岡   | 幸太郎 | 第一営業部 | 担当   |
| 葉山   | 俊輔   | 第一営業部 | アシスタント |
| 妹尾   | 春樹   | 第一営業部 | 課長   |
| 田辺   | 正一   | 第一営業部 | アシスタント |
| 寺岡   | 陽一   | 第一営業部 | 部長   |
| 大門   | 一郎   | 人事部     | 担当   |
| 神田   | 佐知子 | 人事部     | アシスタント |
| 清水   | 春子   | 人事部     | 主任   |
| 遠山   | 真一   | 人事部     | 課長   |
| ... 後略 ...
+-----+-----+-----+-----+
21 rows in set (0.01 sec)
```

社員テーブル (employee : e)

s_id	l_name	f_name	...	class	depart_id	...	retired	...	last_update
AI00001	相沢	聡	...	部長	S02	...	0	...	2012-02-24
DA00001	大門	一郎	...	担当	J01	...	0	...	2011-08-10
FU00001	藤井	雄太	...	主任	E01	...	0	...	2011-04-10
KA00001	川口	裕子	...	アシスタント	S01	...	1	...	2012-12-28
HA00001	速水	和幸	...	部長	E01	...	1	...	2007-02-03

所属部署テーブル (depart : d)

depart_id	depart_name
S01	経営企画部
E01	第一営業部
E02	第二営業部
J01	人事部
S02	総務部

depart_id 列が等しいものを結合

s_id	l_name	f_name	...	class	depart_id	depart_name	...	retired	...	last_update
AI00001	相沢	聡	...	部長	S02	総務部	...	0	...	2012-02-24
TA00002	田辺	正一	...	アシスタント	E01	第一営業部	...	0	...	2012-12-19
FU00001	藤井	雄太	...	主任	E01	第一営業部	...	0	...	2011-04-10
FU00002	藤岡	幸太郎	...	担当	S01	経営企画部	...	1	...	2012-12-28
HA00001	速水	和幸	...	部長	E01	第一営業部	...	1	...	2007-02-03

取得列: l_name, f_name, depart_name, class
retired<>1 (在職) で絞り込み

l_name	f_name	depart_id	s_id	depart_name	class
相沢	聡	S02	AI00001	総務部	部長
田辺	正一	E01	TA00002	第一営業部	アシスタント
藤井	雄太	E01	FU00001	第一営業部	主任

(次ページへ)

2 より複雑な問い合わせを理解しよう

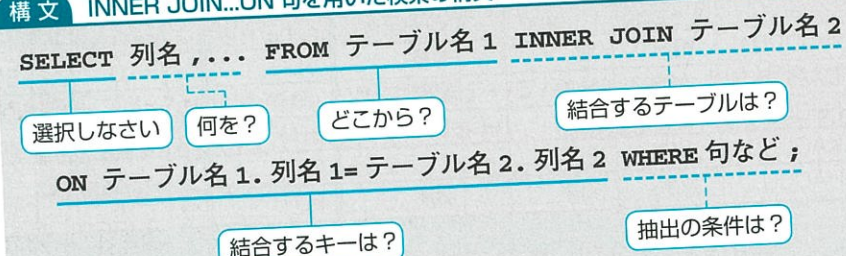
(前ページから) ↓ depart_id, s_id で昇順ソート

l_name	f_name	depart_id	s_id	depart_name	class
藤井	雄太	E01	FU00001	第一営業部	主任
田辺	正一	E01	TA00002	第一営業部	アシスタント
相沢	聡	S02	AI00001	総務部	部長

🔗 構文を理解しよう

複数のテーブルを結合する INNER JOIN...ON 句を利用した検索の構文は、以下の通りです。

構文 INNER JOIN...ON 句を用いた検索の構文



つまり、「<テーブル名 1>と<テーブル名 2>とを<列名 1>と<列名 2>とをキーに関連付け、キーが合致する行のみを選択しなさい」というわけです。もちろん、Chapter1 で学習した WHERE や ORDER BY、GROUP BY などの句を併用することも可能です。

このように、テーブル同士の主キーと外部キーとを結合し、一致する行だけを取り出す結合のことを**内部結合**と言います。後述する「左 (右) 外部結合」と合わせて、**等価結合**という場合もあります (ON 句において、列名を = 演算子で結び付けていることから、こう呼ばれます)。

複数のテーブルを結合する場合、原則として、列名には「テーブル名. 列名」の形式で属しているテーブルを明示的に示す必要があります。このように記述することで、テーブル間で列名が重複した場合にも、競合を未然に防ぐことができます (構文的には、列名の競合がない限り、テーブル名での修飾は必ずしも必須ではありません)。

また、いちいちテーブル名を記述するのが冗長であるという場合には、AS 句でテーブルの別名をあらかじめ用意しておくことも可能です。これによって、SQL 命令をよりシンプルに記述できます。ただし、Oracle ではテーブル名の別名を指定する AS 句を利用できませんので、「テーブル名 別名」のように空白区切りで別名を指定する必要があります。

NOTE

SELECT 命令を実行する場合、データベース側ではまず指定されたテーブルの有無を、次に指定された列の有無を、それぞれ内部的に確認します。

本項の例のように、複数のテーブルを扱う SELECT 命令の場合もそれは同様ですが、この際に列名がテーブル名で修飾されていない場合はどうでしょう。結合に関わるすべてのテーブルを走査し、列が存在するかどうかを確認しなければなりません。しかし、テーブル名を明記することで、指定されたテーブルの中だけを走査すればよいので、いくらか実行のオーバーヘッドを軽減することができます。

SQL 命令の読みやすさはもちろんですが、テーブル名の修飾はパフォーマンスの観点からも重要なポイントです。

🔗 SQL 命令を記述してみよう

INNER JOIN...ON 句を含む SELECT 命令を記述する流れは、以下の通りです。

STEP 1 復習しよう

結合する場合にも、まずは主体となるテーブル単体での条件句を記述してしまい、その上で、結合条件を付与すると、比較的頭の整理がしやすいはずです。

WHERE 句や ORDER BY 句を用いた SQL 命令については学習済みです。ここでは、前項までの復習も兼ねて、ここまで紹介した書き順に沿って、まずは「社員テーブル (employee) から在職の (退職していない) 社員情報を、所属部署コード、社員コードについて昇順で取り出して」みましょう。取り出す列は、社員氏名と役職とします。

2 より複雑な問い合わせを理解しよう

この時点で、以下のような SQL 命令が記述できていれば正解です。続けて、以降のステップを見ていくことにしましょう。

```
SELECT
  l_name,
  f_name,
  class
FROM
  employee
WHERE
  retired <> 1
ORDER BY
  depart_id ASC,
  s_id ASC
;
```

STEP 3 結合するテーブルは？

次に、結合先のテーブルを **INNER JOIN** 句で指定します。また、複数のテーブルを結合する場合、それぞれのテーブルに対して別名を付与しておきましょう。

更に、それぞれの列名をテーブル名の別名で修飾しておきます。「テーブルの別名.列名」の形式です。

```
SELECT
  e.l_name,
  e.f_name,
  e.class
FROM
  employee AS e
INNER JOIN
  depart AS d
WHERE
  e.retired <> 1
ORDER BY
  e.depart_id ASC,
  e.s_id ASC
;
```

STEP 4 結合のキーは？

等価結合する場合、結合するキー同士（主キーと外部キー）を **ON** 句で指定する必要があります。なお、結合するキー同士は同じデータ型である必要がありますが、列名自体は必ずしも一致している必要はありません。

```
SELECT
  e.l_name,
  e.f_name,
  e.class
FROM
  employee AS e
INNER JOIN
  depart AS d
ON
  e.depart_id=d.depart_id
WHERE
  e.retired <> 1
ORDER BY
  e.depart_id ASC,
  e.s_id ASC
;
```

STEP 4 結合先のテーブルから取得する列は？

最後に結合先のテーブル（ここでは所属部署テーブル「depart」）から取得する列を追加します。

```
SELECT
  e.l_name,
  e.f_name,
  d.depart_name,
  e.class
FROM
  employee AS e
INNER JOIN
  depart AS d
ON
  e.depart_id=d.depart_id
WHERE
  e.retired <> 1
ORDER BY
  e.depart_id ASC,
  e.s_id ASC
;
```

完成!

これで1つの SQL 命令が完成です。

NOTE

内部結合は、INNER JOIN...ON 句を使わず、WHERE 句だけでも表現できます。たとえば、例題の SQL 命令を WHERE 句で置き換えてみると、以下のようになります。

```
SELECT
  e.l_name,
  e.f_name,
  d.depart_name,
  e.class
FROM
  employee AS e,
  depart AS d
WHERE
  e.depart_id=d.depart_id
AND
  e.retired <> 1
ORDER BY
  e.depart_id ASC,
  e.s_id ASC
;
```

FROM 句に結合する 2 つのテーブルを、WHERE 句に結合条件を記述するわけです。ただし、WHERE 句による結合条件の記述は、多くのデータベース製品において、外部結合（後述）では利用できません。また、いわゆる普通の絞り込み条件と結合条件とを区別する意味でも、INNER JOIN...ON 句を利用した方が分かりやすいので、基本的には INNER JOIN...ON 句を優先して利用することをお勧めします。

マスタ・プラクティス



問 1

社員テーブル (employee) とタイムカードテーブル (time_card) から氏名と2012年12月の勤務時間を、社員コード、日付について昇順で出力してみましょう。空欄を埋めて、SQL命令を完成させましょう。

```
SELECT
  ①
FROM
  employee AS e
  ②
  time_card AS t
ON
  ③
WHERE
  t.r_date ④ '2012-12-01' ⑤ '2012-12-31'
ORDER BY
  e.s_id ASC,
  t.r_date ASC
;
```

問 2

店舗テーブル (shop) と月間売り上げテーブル (sales) から2012年12月の売り上げを売上高の高い順に出力してみましょう。

```
SELECT
  sh.s_name,
  sl.s_value
FROM
  ①
INNER JOIN
  ②
ON
  ③
WHERE
  ④
ORDER BY
  ⑤
;
```


問 3

店舗テーブル (shop) と月間売り上げテーブル (sales) から2012年の全売上高を、売上高の低い順に出力してみましょう。

問 4

貸し出し記録テーブル (rental) とユーザテーブル (usr) からそれぞれのユーザについて、現在何冊を貸し出しているかを、貸し出し数の多い順に取得してみましょう。取得列は、ユーザの氏名、貸し出し数とします。また、貸し出し数0のユーザは出力する必要はありません。

2-2

外部結合で複数テーブルのデータを結合したい

RIGHT JOIN...ON 句 / LEFT JOIN...ON 句

MySQL PostgreSQL SQLite SQL Server Oracle



前項で学習した「内部結合」という言葉に対して、「外部結合」という概念があります。外部結合を利用することで、結合する片方のテーブルにしかない情報も取り出すことができます。



注文明細テーブル (order_desc) と商品テーブル (product) とから、商品ごとの累計購入数と累計購入額を、購入額が多い順に取り出してみましょう。なお、まったく売れていない商品についても取り出すものとします。



片方のテーブル（この場合は、注文明細テーブル）に存在しないデータも含めて取り出したいという場合には、RIGHT JOIN...ON 句を使用します。

```
> SELECT
>   p.p_name,
>   SUM(o.quantity),
>   SUM(p.price * o.quantity)
> FROM
>   order_desc AS o
> RIGHT JOIN
>   product AS p
> ON
>   p.p_id = o.p_id
> GROUP BY
>   p.p_id,
>   p.p_name
> ORDER BY
>   SUM(p.price * o.quantity) DESC
> ;
```

p_name	SUM(o.quantity)	SUM(p.price * o.quantity)
赤ボールペン	600	60000
電卓	90	54000
黒ボールペン	401	40100
机	2	30000

椅子	3	30000
トナー黒	20	20000
トナー赤	20	20000
マウスパッド	NULL	NULL

8 rows in set (0.06 sec)

注文明細テーブル (order_desc : o)

po_id	p_id	quantity
3	DE000000001	20
4	T0000000001	10
5	T0000000002	10
11	T0000000001	5
16	T0000000001	5

商品テーブル (product : p)

p_id	p_name	price
PB000000001	黒ボールペン	100
PB000000002	赤ボールペン	100
DE000000001	電卓	600
T0000000001	トナー黒	1000
T0000000002	トナー赤	1000

p_id 列が合致する行、及び、
商品テーブルのすべてを取得

po_id	o.p_id	p.p_id	quantity	p_name	price
3	DE000000001	DE000000001	20	電卓	600
4	T0000000001	T0000000001	10	トナー黒	1000
5	T0000000002	T0000000002	10	トナー赤	1000
11	T0000000001	T0000000001	5	トナー黒	1000
16	T0000000001	T0000000001	5	トナー黒	1000
NULL	NULL	PB000000001	NULL	黒ボールペン	100
NULL	NULL	PB000000002	NULL	赤ボールペン	100

グループ化キー : p.p_id, p.p_name
集計列 : SUM(o.quantity), SUM(p.price * o.quantity)

p_name	SUM (quantity)	SUM (price*quantity)
黒ボールペン	NULL	NULL
赤ボールペン	NULL	NULL
電卓	20	12000
トナー黒	20	20000
トナー赤	10	10000

SUM(p.price * o.quantity) で降順ソート

p_name	SUM (quantity)	SUM (price*quantity)
トナー黒	20	20000
電卓	20	12000
トナー赤	10	10000
黒ボールペン	NULL	NULL
赤ボールペン	NULL	NULL

構文を理解しよう

複数のテーブルを結合する **RIGHT JOIN...ON** 句を利用した検索の構文は、以下の通りです。

構文 **RIGHT JOIN...ON** 句を用いた検索の構文 (右外部結合)

SELECT 列名, ... **FROM** テーブル名 1 **RIGHT JOIN** テーブル名 2

選択しなさい 何を? どこから? 結合するテーブルは?

ON テーブル名 1. 列名 1= テーブル名 2. 列名 2 **WHERE** 句など;

結合するキーは? どのような条件で

構文そのものは内部結合 (INNER JOIN...ON 句) と同様ですので、比較的分かりやすいのではないのでしょうか。つまり、「<テーブル名 1>と<テーブル名 2>とを<列名 1>と<列名 2>とをキーに関連付け、キーが合致する行、及び、<テーブル名 2>に含まれるすべての行を選択しなさい」というわけです。もちろん、Chapter1 で学習した WHERE や ORDER BY、GROUP BY などの句を併用することも可能です。

このように、右テーブルの内容すべてと左テーブルの関連付けられた値のみを取り出す結合のことを**右外部結合**といいます。逆に、左テーブルの内容すべてと右テーブルの関連付けられた値のみを取り出す結合のことを**左外部結合**といいます。左外部結合は **LEFT JOIN...ON** 句を利用し、構文は以下の通りです。

構文 **LEFT JOIN...ON** 句を用いた検索の構文 (左外部結合)

SELECT 列名, ... **FROM** テーブル名 1 **LEFT JOIN** テーブル名 2

選択しなさい 何を? どこから? 結合するテーブルは?

ON テーブル名 1. 列名 1= テーブル名 2. 列名 2 **WHERE** 句など;

結合するキーは? どのような条件で

また、左外部結合と右外部結合とを合わせて、**外部結合**とも言います。「LEFT JOIN」「RIGHT JOIN」の代わりに、「LEFT OUTER JOIN」「RIGHT OUTER JOIN」と記述してもかまいません。

さて、ここで頭が混乱してきた読者の方もいらっしゃると思いますので、内部結合と左 (右) 外部結合について、整理してみることになしましょう。

2 より複雑な問い合わせを理解しよう

●内部結合

注文テーブル	
注文番号	商品番号
00001	A-01
00002	B-01
00003	C-01
00004	A-01
00005	B-01

商品テーブル	
商品番号	商品名
A-01	りんご
B-01	ハムスターの小屋
Z-00	自転車

合致した行のみを結合

注文番号	商品番号	商品名
00001	A-01	りんご
00002	B-01	ハムスターの小屋
00004	A-01	りんご
00005	B-01	ハムスターの小屋

●左外部結合

注文テーブル	
注文番号	商品番号
00001	A-01
00002	B-01
00003	C-01
00004	A-01
00005	B-01

商品テーブル	
商品番号	商品名
A-01	りんご
B-01	ハムスターの小屋
Z-00	自転車

合致した行と左テーブルのすべてを結合

注文番号	商品番号	商品名
00001	A-01	りんご
00002	B-01	ハムスターの小屋
00003	C-01	
00004	A-01	りんご
00005	B-01	ハムスターの小屋

●右外部結合

注文テーブル	
注文番号	商品番号
00001	A-01
00002	B-01
00003	C-01
00004	A-01
00005	B-01

商品テーブル	
商品番号	商品名
A-01	りんご
B-01	ハムスターの小屋
Z-00	自転車

合致した行と右テーブルのすべてを結合

注文番号	商品番号	商品名
00001	A-01	りんご
00002	B-01	ハムスターの小屋
00004	A-01	りんご
00005	B-01	ハムスターの小屋
	Z-00	自転車

いかがですか？

内部結合では2つのテーブル間でキーが一致した行だけが取り出されるのに対して、左（右）外部結合では、一致の如何に関わらず、左（右）のテーブルの内容はすべて取り出されるのが大きな違いです。

なお、左外部結合と右外部結合とは、基準となるテーブルが左か右かというだけで、本質的な違いはありません。アプリケーション内では混在させるのではなく、いずれか片方を利用するようにした方がコードの可読性を維持できます。

ただし、SQLite では、RIGHT JOIN 句に対応していません。左右のテーブルを入れ替えて、LEFT JOIN 句を使用してください。例題の場合は、以下のようになります。

```
SELECT
  p.p_name,
  SUM(o.quantity),
  SUM(p.price * o.quantity)
FROM
  product AS p
```

```
LEFT JOIN
  order_desc AS o
ON
  o.p_id = p.p_id
GROUP BY
  p.p_id,
  p.p_name
ORDER BY
  SUM(p.price * o.quantity) DESC
;
```

NOTE

結合条件を表す双方テーブルの列名が同名である場合には、ON 句の代わりに、USING 句を利用することも可能です。

```
ON p.p_id = o.p_id ⇔ USING(p_id)
```

USING 句は便利な構文ですが、Oracle や SQL Server などのデータベースでは利用できませんので、本書では原則、ON 句を優先して使用しています。



SQL 命令を記述してみよう

LEFT JOIN...ON 句を含む SELECT 命令を記述する流れは、以下の通りです。

STEP 1 復習しよう

前項でも紹介したように、複数のテーブルを扱う場合も、まずは主体となるテーブル単体の条件句を記述してしまうことです。その上で、結合条件や伴う条件句を付与すると、複数のテーブルを扱う場合にも、比較的、頭を整理しやすくなります。

そこで、ここではまず前項までの復習も兼ねて、「商品テーブル (product) から登録されているすべての商品名を取り出して」みましょう。

```
SELECT
  p.p_name,
  SUM(o.quantity),
  SUM(p.price * o.quantity)
FROM
  product AS p
```


2 より複雑な問い合わせを理解しよう

この時点で、以下のような SQL 命令が記述できていれば正解です。続けて、以降のステップを見ていくことにしましょう。

```
SELECT
  p_name
FROM
  product
;
```

STEP2 結合テーブルとそのキーは？

次に、結合先のテーブルを指定します。この際、注文明細テーブルとは合致しない商品テーブルの内容も取り出したいので、(右) 外部結合を表す「RIGHT JOIN...ON」句で結合条件を指定します。

また、複数テーブルを扱う場合には、それぞれの列名をテーブル名の別名で修飾しておきます。「テーブルの別名.列名」の形式です。

```
SELECT
  p.p_name
FROM
  order_desc AS o
RIGHT JOIN
  product AS p
ON
  p.p_id = o.p_id
;
```

STEP3 グループニングの条件は？

結合先のテーブルの内容に基づいて、グループ化キーを指定します。ここでは、商品コードについてグループニングします。ただし、取得列に商品名が含まれていますので、グループ化キーにも商品名 (p_name) を追加している点に注意してください。

```
SELECT
  p.p_name
FROM
  order_desc AS o
RIGHT JOIN
  product AS p
ON
  p.p_id = o.p_id
GROUP BY
  p.p_id,
```

```
p.p_name
;
```

STEP4 取得列は？

最後に結合先のテーブル (ここでは注文明細テーブル「order_desc」) から取得する列とソート条件を追加します。

```
SELECT
  p.p_name,
  SUM(o.quantity),
  SUM(p.price * o.quantity)
FROM
  order_desc AS o
RIGHT JOIN
  product AS p
ON
  p.p_id = o.p_id
GROUP BY
  p.p_id,
  p.p_name
ORDER BY
  SUM(p.price * o.quantity) DESC
;
```

完成!

これで1つの SQL 命令が完成です。

NOTE

本問の場合、単純にグループ化という目的を考えた場合、GROUP BY 句に p_name 列を含む必要はありません。p_id 列で商品ごとのグループ化は果たせるからです。にもかかわらず、ここで p_name 列をグループ化キーに含めているのは、「GROUP BY 句を使用した場合、グループ化キー、または集計キーでない列を出力できない」という制約を回避するためです。このような記述を回避したい場合には、以下のように無理やり対象列（この場合は、p_name 列）を集計列にしてしまうことも可能です。

```
SELECT
  MAX(p.p_name),
  SUM(o.quantity),
  SUM(p.price * o.quantity)
FROM
  order_desc AS o
RIGHT JOIN
  product AS p
ON
  o.p_id = p.p_id
GROUP BY
  p.p_id
ORDER BY
  SUM(p.price * o.quantity) DESC
;
```

いずれの記法がより良いとは一概には言えませんが、1つの書き方として覚えておくといでしょう。

マスタ・プラクティス



問 1

ユーザテーブル (usr) と貸し出し記録テーブル (rental) から、ユーザごとの貸し出し数を、貸し出し数が多い順に取得してみましょう。取得列は、ユーザの氏名と貸し出し件数とし、貸し出し件数が0のユーザも出力するものとします。以下の空欄を埋めて、SQL命令を完成させましょう。

```
SELECT
  u.l_name,
  u.f_name,
  COUNT(r.id)
FROM
  rental AS r
  ①
  usr AS u
ON
  ②
GROUP BY
  ③
ORDER BY
  ④
;
```

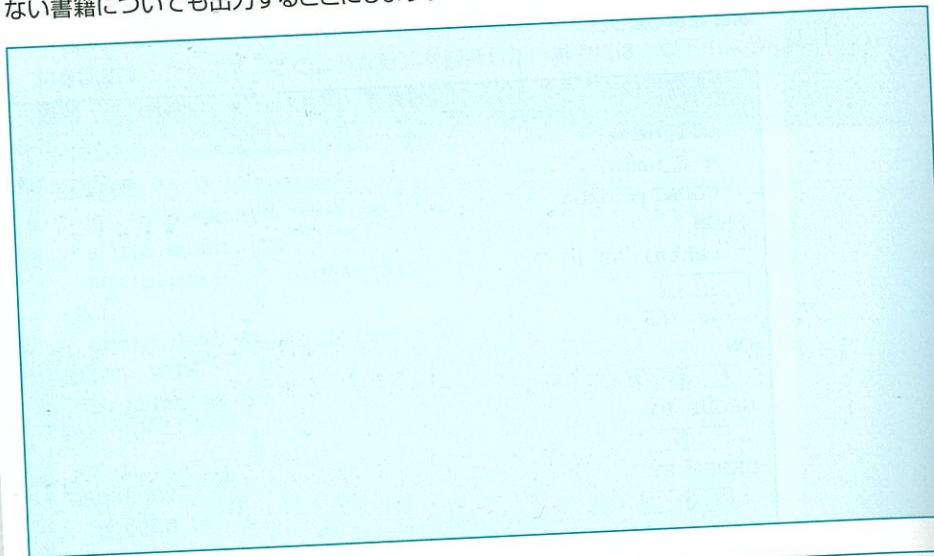
問 2

社員テーブル (employee) と所属部署テーブル (depart) を結合し、所属社員が一人もいない「幽霊」部署を洗い出してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```
SELECT
  d.depart_id,
  d.depart_name
FROM
  depart AS d
  ①
  employee AS e
ON
  ②
WHERE
  ③
;
```

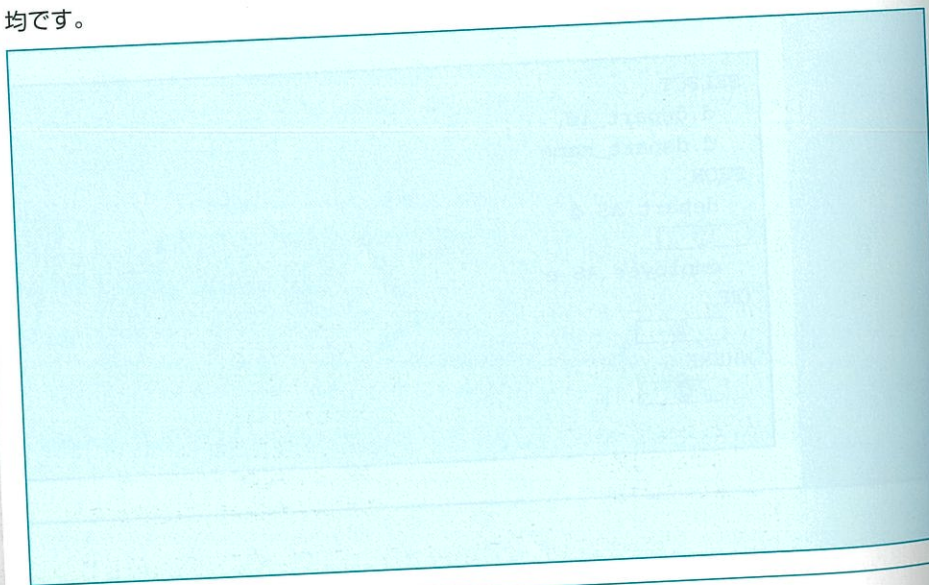

問 3

書籍情報テーブル (books) と貸し出し記録テーブル (rental) とを結合し、書籍ごとの貸し出し数累計を、累計数が多い順に出力してみましょう。その際、貸し出し履歴が1回もない書籍についても出力することになります。



問 4

社員テーブル (employee) とタイムカードテーブル (time_card) を結合し、社員ごとの勤務時間平均を、平均時間の多い順に出力してみましょう。その際、タイムカードへの記録がない社員の情報についても出力することになります。出力列は、社員氏名、勤務時間平均です。



2-3 自己結合で同一テーブル内のデータを結合したい JOIN 句 + 別名定義

MySQL PostgreSQL SQLite SQL Server Oracle



ここまでは、複数のテーブルを結合する「内部結合」「外部結合」という概念を学んできましたが、結合は必ずしも複数のテーブルを対象にしたものばかりではありません。同じテーブルに別名をつけて、あたかも異なるテーブルであるかのように結合することも可能です。このような結合のことを「自己結合」といいます。

Q 書籍目次テーブル (contents) から現在のコンテンツ名と次のコンテンツ名を、コンテンツコードについて昇順で取り出してみましょう。

A 自己結合は扱うのが同一のテーブル同士というだけで、内部結合と同様、INNER JOIN...ON 句で表現できます。

```
> SELECT
>   cc.c_title,
>   cn.c_title
> FROM
>   contents AS cc
> INNER JOIN
>   contents AS cn
> ON
>   cc.next_id = cn.c_id
> ORDER BY
>   cc.c_id ASC
> ;
```

c_title	c_title
SQL について	データの取得 1
データの取得 1	カラムの選択
カラムの選択	WHERE 句の記述
WHERE 句の記述	NULL と空白の扱い
NULL と空白の扱い	あいまい検索
あいまい検索	複数条件の連結 (論理演算子)

複数条件の連結 (論理演算子)
 重複レコードの除去
 データの取得 2
 順位付け
 ... 後略 ...

重複レコードの除去
 データの取得 2
 順位付け
 ソート

29 rows in set (0.00 sec)

書籍目次テーブル (contents : cc)		
c_id	c_title	next_id
A001	SQL について	A011
A012	カラムの選択	A013
A011	データの取得 1	A012
A013	WHERE 句の記述	A014
...	...	...

書籍目次テーブル (contents : cn)		
c_id	c_title	next_id
A001	SQL について	A011
A012	カラムの選択	A013
A011	データの取得 1	A012
A013	WHERE 句の記述	A014
...	...	...

next_id 列と c_id 列の
 等しいものを結びつけ

cc.c_id	cc.c_title	cc.next_id	cn.c_id	cn.c_title	cn.next_id
A001	SQL について	A011	A011	データの取得 1	A012
A012	カラムの選択	A013	A013	WHERE 句の記述	A014
A011	データの取得 1	A012	A012	カラムの選択	A013
A013	WHERE 句の記述	A014	A014	NULL と空白の扱い	A015
...	...	...	...	...	...

指定された列を取り出し

cc.c_id	cc.c_title	cn.c_title
A001	SQL について	データの取得 1
A012	カラムの選択	WHERE 句の記述
A011	データの取得 1	カラムの選択
A013	WHERE 句の記述	NULL と空白の扱い
...	...	...

cc.c_id 列で昇順ソート

cc.c_id	cc.c_title	cn.c_title
A001	SQL について	データの取得 1
A011	データの取得 1	カラムの選択
A012	カラムの選択	WHERE 句の記述
A013	WHERE 句の記述	NULL と空白の扱い
...	...	...

構文を理解しよう

自己結合といっても、なにも特別なことはありません。ただ、(異なるテーブル同士ではなく) 同一テーブル同士で扱うというだけで、構文自体は通常の内部結合、または外部結合と同様、INNER (LEFT、RIGHT) JOIN..ON 句を利用できます。

唯一、これまでの内部結合、外部結合と異なる点は、自己結合の場合には、テーブルに「別名をつけることが必須である」という点です (内部結合や外部結合では、別名定義が「好ましい」というだけで必須ではありませんでした)。たとえば、以下は自己結合で別名を利用しなかった場合の例です。

```
SELECT
  contents.c_title,
  contents.c_title
FROM
  contents
INNER JOIN
  contents
ON
  contents.next_id = contents.c_id
ORDER BY
  contents.c_id ASC
;
```

いかがですか？ これでは、結合元のテーブルと結合先のテーブルとを判別することができません。自己結合では、テーブルに別名をつけて2つのテーブルを「あたかも異なるテーブルであるかのように」見なすことが重要なのです。

もちろん、ここまで内部結合や外部結合でもきちんと別名定義をしてきた本書読者の皆さんならば問題ないと思いますが、今一度、ここで注意してください。

NOTE

ここまで、等価結合 (内部結合、左/右外部結合)、自己結合などを紹介してきましたが、その他にも、左外部結合と右外部結合とを合わせた外部結合 (FULL OUTER JOIN)、テーブル1の全行に対してテーブル2の全行をすべての組み合わせで結合する交差結合 (CROSS JOIN) などもあります。

これらの結合については、比較的、利用頻度も少ないと思われるので (特に交差結合はその性質上、結果行数が膨大になる可能性があり、テスト用データの作成など特殊な用途を除いては利用すべきではありません)、本書では割愛します。詳細については、それぞれのデータベース製品のドキュメントなどを参照してください。

SQL 命令を記述してみよう

自己結合を利用した SELECT 命令の記述方法は、以下の通りです。

STEP 1 復習しよう

繰り返しますが、自己結合といっても、扱うテーブルが自分自身（同一のテーブル）というだけで、これまでに学んできた結合の方法と変わることはありません。結合する場合の基本は、まず単体テーブルとしての SQL を記述してしまい、その後でおもむろにテーブルを結合するということでした。

そこで、ここでは（もう当たり前の基本的な SQL 命令ですが）、「書籍目次テーブル（contents）からコンテンツコードについて昇順で情報を取り出して」みましょう。

この時点で、以下のような SQL 命令が記述できていれば正解です。続けて、以降のステップを見ていくことにしましょう。

```
SELECT
  c_title
FROM
  contents
ORDER BY
  c_id ASC
;
```

STEP 2 結合テーブルとそのキーは？

次に、INNER JOIN 句で結合先のテーブルを指定します。自己結合では、結合先のテーブルも自分自身ですので、両者を区別するためにそれぞれのテーブル名に対して、「必ず」別名を定義しておきます。また、これに合わせて、それぞれの列名もテーブル別名で修飾しておきましょう。

```
SELECT
  cc.c_title
FROM
  contents AS cc
INNER JOIN
  contents AS cn
ON
  cc.next_id = cn.c_id
ORDER BY
  cc.c_id ASC
;
```

STEP 3 結合先テーブルから取得する列は？

最後に、結合先テーブル（自分自身）から取得する列を追加して完成です。

```
SELECT
  cc.c_title,
  cn.c_title
FROM
  contents AS cc
INNER JOIN
  contents AS cn
ON
  cc.next_id = cn.c_id
ORDER BY
  cc.c_id ASC
;
```

完成！

これで1つの SQL 命令が完成です。

マスタ・プラクティス

問 1

書籍目次テーブル (contents) から現在のコンテンツコードと前のコンテンツコードを、現在のコンテンツコードについて昇順で取り出してみましょう。以下の空欄を埋めて、SQL 命令を完成させてください。

```
SELECT
  cp.c_id AS 前のページ,
  cn.c_id AS 現在のページ
FROM
  ①
INNER JOIN
  contents AS cn
ON
  ②
ORDER BY
  ③
;
```

問 2

メニューテーブル (menu) からそれぞれのメニューに対応する親メニュー名をメニューコードについて昇順に取り出してみましょう。以下の空欄を埋めて、SQL 命令を完成させてください。

```
SELECT
  ① AS メニュー名,
  ② AS 親メニュー名
FROM
  ③ AS mc
INNER JOIN
  ④ AS mn
ON
  ⑤
ORDER BY
  ⑥
;
```

問 3

社員テーブル (employee) からそれぞれの社員氏名と上司氏名を、社員コードについて昇順に取り出してみましょう。なお、取り出す列には別名として「社員氏名」「上司氏名」とし、上司がいない社員についても情報は取り出すことにします。

```
SELECT
  emp.c_id AS 社員氏名,
  emp.c_id AS 上司氏名
FROM
  employee AS emp
  ①
ON
  ②
ORDER BY
  ③
;
```

問 4

以下は、書籍目次テーブル (contents) から現在のコンテンツコードと前のコンテンツコードを、現在のコンテンツコードについて昇順で取り出すための SQL ですが、誤りが2点あります。誤りを指摘してください。

```
SELECT
  cp.c_id AS 前のページ,
  cn.c_id AS 現在のページ
FROM
  contents AS cp
OUTER JOIN
  contents AS cn
ON
  cp.next_id <> cn.c_id
ORDER BY
  cn.c_id ASC
;
```


2-4

3つ以上のテーブルを結合したい
(結合の応用) JOIN 句の入れ子

MySQL PostgreSQL SQLite SQL Server Oracle



結合できるのはなにも2つのテーブルばかりではありません。INNER (LEFT、RIGHT) JOIN...ON 句を入れ子に利用することで、3つ以上のテーブルを結合することも可能です。



書籍情報テーブル (books)、著者-書籍情報テーブル (author_books)、著者情報テーブル (author) を結合して、出版社が「日経 BP」である書籍情報を刊行日の新しい順に取り出してみましょう。取得列は、書名、著者名、刊行日とします。



3つ以上のテーブル結合は、JOIN 句を入れ子に利用することで実現できます。

```
> SELECT
>   b.title,
>   a.name,
>   b.publish_date
> FROM
>   (
>     books AS b
>     INNER JOIN
>       author_books AS ab
>       ON
>         b.isbn = ab.isbn
>   )
>   INNER JOIN
>     author AS a
>     ON
>       ab.author_id = a.author_id
> WHERE
>   b.publish='日経 BP'
> ORDER BY
>   b.publish_date DESC
> ;
```

title	name	publish_date
SQL リファレンス	佐藤一郎	2013-02-15

2-4 3つ以上のテーブルを結合したい (結合の応用)

XML リファレンス	田中太郎	2012-11-24
SQL プチブック	川中智樹	2010-11-30

+-----+-----+-----+
> 3 rows in set (0.08 sec)

書籍情報テーブル (books : b)

isbn	title	publish	...	publish_date
4-0010-0000-0	ハムスターの観察	山田出版	...	2010-11-01
4-7981-0959-2	PEAR 入門	翔泳社	...	2012-09-08
4-7980-0945-8	PHP4 サンプル集	秀和システム	...	2012-11-01
4-8833-0000-2	SQL プチブック	日経 BP	...	2010-11-30
4-8833-0000-1	SQL リファレンス	日経 BP	...	2013-02-15

isbn 列の等しい行を結合

著者-書籍情報テーブル (author_books : ab)

isbn	author_id
4-0010-0000-0	Y0001
4-7981-0959-2	Y0002
4-7980-0945-8	Y0002
4-8833-0000-2	K0001
4-8833-0000-1	S0001

isbn	author_id	title	publish	...	publish_date
4-0010-0000-0	Y0001	ハムスターの観察	山田出版	...	2010-11-01
4-7981-0959-2	Y0002	PEAR 入門	翔泳社	...	2012-09-08
4-7980-0945-8	Y0002	PHP5 サンプル集	秀和システム	...	2012-11-01
4-8833-0000-2	S0001	SQL プチブック	日経 BP	...	2010-11-30
4-8833-0000-1	K0001	SQL リファレンス	日経 BP	...	2013-02-15

author_id 列の等しい行を統合

著者情報テーブル (author : a)

author_id	name	birth
Y0001	山田愛子	1970-04-06
S0001	佐藤一郎	1973-05-06
T0001	田中太郎	1975-11-01
K0001	川中智樹	1986-03-12
Y0002	山田祥寛	1960-12-04

isbn	author_id	title	publish	...	publish_date	name	birth
4-0010-0000-0	Y0001	ハムスターの観察	山田出版	...	2010-11-01	山田愛子	1970-04-06
4-7981-0959-2	Y0002	PEAR 入門	翔泳社	...	2012-09-08	山田祥寛	1960-12-04
4-7980-0945-8	Y0002	PHP5 サンプル集	秀和システム	...	2012-11-01	山田祥寛	1960-12-04
4-8833-0000-2	S0001	SQL プチブック	日経 BP	...	2010-11-30	佐藤一郎	1986-03-12
4-8833-0000-1	K0001	SQL リファレンス	日経 BP	...	2013-02-15	川中智樹	1973-05-06

(次ページへ)

(前ページから)

b.publish='日経 BP' で絞り込み

isbn	author_id	title	publish	...	publish_date	name	birth
4-8833-0000-2	S0001	SQL プチブック	日経 BP	...	2010-11-30	佐藤一郎	1986-03-12
4-8833-0000-1	K0001	SQL リファレンス	日経 BP	...	2013-02-15	川中智樹	1973-05-06

title	name	publish_date
SQL プチブック	佐藤一郎	2010-11-30
SQL リファレンス	川中智樹	2013-02-15

title	name	publish_date
SQL リファレンス	川中智樹	2013-02-15
SQL プチブック	佐藤一郎	2010-11-30

 構文を理解しよう

入れ子の INNER JOIN 句を記述する基本的な構文は、以下の通りです。ここでは、内部結合する場合を想定していますが、もちろん、外部結合を用いる場合には INNER を LEFT (RIGHT) に置き換えるだけで、あとは同様の記述が可能です。

構文 入れ子の JOIN 句を用いた構文

SELECT 列名,... FROM (テーブル名 1

選択しなさい 何を? どこから?

INNER JOIN テーブル名 2 ON テーブル名 1. 列名 1= テーブル名 2. 列名 1)

結合するテーブルは？

結合するキーは？

INNER JOIN テーブル名3 ON テーブル名2.列名21= テーブル名3.列名1 WHERE 句など;

結合するテーブルは？

結合するキーは？

抽出の条件は？

いかがですか？

いかがですか？

2つのテーブルを結合するところまでは、前項までに学んだ内容と変わるところはありません。3つ以上のテーブルを結合する場合には、この結合条件全体を1つのテーブルと見なして、更に INNER JOIN 句で内部結合するわけです。

ここで紹介しているのは3つのテーブルを結合するまでですが、もちろん、4つ以上のテーブルを結合する場合も、JOIN 句を連ねていくことで可能です。

SQL 命令を記述してみよう

3つ以上のテーブルを結合する SQL 命令の記述方法は、以下の通りです。

STEP 1 復習しよう

繰り返しますが、3つ以上のテーブルを結合する場合も、SQL 記述の基本的な流れは変わりません。まずは単体のテーブルによる単純な SQL 命令を記述した後、必要な結合条件を加えていきます。

ここでは、前項までの復習も兼ねて、まずは「書籍情報テーブル (books) から出版社が「日経 BP」である書籍情報を刊行日の新しい順に取り出して」みましょう。取り出す列は、書名と刊行日とします。

[illegible]

この時点で、以下のような SQL 命令が記述できていれば正解です。続けて、以降のステップを見ていくことにしましょう。

```
SELECT
    title,
    publish_date
FROM
    books
WHERE
    publish='日経BP'
ORDER BY
    publish_date DESC
;
```

STEP 2 第1の結合テーブルとそのキーは？

次に、前項同様の要領で1番目の結合テーブルとその結合条件を追加してみましょう。

2 より複雑な問い合わせを理解しよう

また、結合する場合のお約束として、テーブル名に別名をつけ、列名もテーブル別名で修飾しておきます。

```
SELECT
  b.title,
  b.publish_date
FROM
  books AS b
INNER JOIN
  author_books AS ab
ON
  b.isbn = ab.isbn
WHERE
  b.publish='日経BP'
ORDER BY
  b.publish_date DESC
;
```

STEP 3 第2の結合テーブルとそのキーは？

更に、2番目の結合テーブルとその結合条件を追加します。追加の要領はSTEP 2と同様で、INNER JOIN...ON 句全体を1つのテーブルと見なし、これに対して、続けてINNER JOIN...ON 句を追加します。

```
SELECT
  b.title,
  b.publish_date
FROM
  (
    books AS b
    INNER JOIN
      author_books AS ab
    ON
      b.isbn = ab.isbn
  )
INNER JOIN
  author AS a
ON
  ab.author_id = a.author_id
WHERE
  b.publish='日経BP'
ORDER BY
  b.publish_date DESC
;
```

STEP 4 結合先のテーブルから取得する列は？

最後に、結合先のテーブル（ここでは著者情報テーブル author）から取得する列を追加します。

```
SELECT
  b.title,
  a.name,
  b.publish_date
FROM
  (
    books AS b
    INNER JOIN
      author_books AS ab
    ON
      b.isbn = ab.isbn
  )
INNER JOIN
  author AS a
ON
  ab.author_id = a.author_id
WHERE
  b.publish='日経BP'
ORDER BY
  b.publish_date DESC
;
```

完成！

これで1つのSQL命令が完成です。

NOTE

ストアドプロシージャ（Stored Procedure）とは、その名の通り、データベースサーバ上に蓄積（Stored）された一連の手続き（Procedure）のことをいいます。ストアドプロシージャはデータベース上で解析／コンパイル済みの状態で用意されていますので、実行時には直接実行でき、普通にアプリケーションからSQL命令を発行するよりも高速な動作が望めます。

ストアドプロシージャでは条件分岐や繰り返し処理など、一般的なプログラミング言語が持つ制御構造を記述可能ですが、データベースによってその構文規則は異なります。OracleではPL/SQLという言語仕様を利用するのが一般的ですし、SQL ServerではTransact SQLを使用します。また、SQL Server 2012ではVisual BasicやC#のようなプログラミング言語を利用してストアドプロシージャを記述できる「SQL CLR」のような仕様も提供されています。

本書では、これら多岐にわたるストアドプロシージャの構文を紹介することはできませんが、データベースを利用する上で欠かせない重要な機能の1つです。次のステップとして、それぞれのデータベース製品ごとの専門書で詳細を学んでみてはどうでしょう。

マスタ・プラクティス

問 1

社員テーブル (employee) と所属部署テーブル (depart)、タイムカードテーブル (time_card) を結合し、社員コード「DA00001」における2012年12月分の勤務時間を日付について昇順に出力してみましょう。出力列の別名は、先頭から「所属部署名」「社員氏名」「勤務時間」とします。

```
SELECT
  ①
FROM
  (
    employee AS e
    ②
    depart AS d
  )
  ON
    ③
  ②
  time_card AS t
  ON
    ④
WHERE
  e.s_id='DA00001'
AND
  t.r_date BETWEEN '2012-12-01' AND '2012-12-31'
ORDER BY
  ⑤
;
```

問 2

書籍目次テーブル (contents) から前後のコンテンツを取り出してみましょう。なお、取り出す列は別名として「前コンテンツ名」「現コンテンツ名」「次コンテンツ名」とし、現ページのコンテンツコードについて昇順に並べるものとします。以下の空欄を埋めて、SQL命令を完成させましょう。

```
SELECT
  ①
FROM
  (
    contents AS cp
    ②
```

```
contents AS cc
ON
  ③
)
  ②
  contents AS cn
  ON
    ④
  ORDER BY
    ⑤
;
```

問 3

注文書テーブル (order_main) と注文明細テーブル (order_desc)、ユーザテーブル (usr)、商品テーブル (product) を結合し、未納の注文について、発注日、注文コード、商品コード昇順に注文情報を出力してみましょう。出力列の別名は、発注日、注文コード、利用者氏名、商品名、商品単価、購入数とします。

2-5 ある問い合わせ結果に基づいて、検索したい サブクエリ

MySQL PostgreSQL SQLite SQL Server Oracle



サブクエリとは、ある問い合わせの結果に基づいて、異なる問い合わせを行うしくみのことをいいます。入れ子になった SELECT 命令だと思っていただいても良いでしょう。

Q 月間売り上げテーブル (sales) に 2012 年 12 月分のデータが登録されて「いない」店舗の店舗コードと、店舗名を店舗テーブル (shop) から取り出してみましょう。

A NOT IN 演算子とサブクエリを使うことで可能です。

```
> SELECT
>   s_id,
>   s_name
> FROM
>   shop
> WHERE
>   s_id
> NOT IN
>   (
>     SELECT
>       s_id
>     FROM
>       sales
>     WHERE
>       s_date='2012-12'
>   )
> ;
```

```
+-----+-----+
| s_id | s_name |
+-----+-----+
| M0002 | 三吉町南店 |
+-----+-----+
1 rows in set (0.00 sec)
```

月間売り上げテーブル (sales)		
s_id	s_date	s_value
K0001	2012-12	11234
M0001	2012-12	11145
K0002	2012-12	10456
H0001	2012-12	9861
M0002	2012-11	12032

2012 年 12 月分のデータだけを抽出 (サブクエリ)

抽出結果にないものだけを抽出	
s_id	s_name
K0001	
M0001	
K0002	
H0001	

s_id	s_name
M0002	三吉町南店

構文を理解しよう

サブクエリは日本語で「副問い合わせ」とも言われる通り、本来の問い合わせを行うための補助的な問い合わせのことを言います。サブクエリにはさまざまな構文がありますが、本項の例文では WHERE 句の条件値をサブクエリで求めています。

構文 サブクエリ (WHERE 句)

SELECT 列名, ... FROM テーブル名 WHERE 列名 演算子 (サブクエリ);

選択しなさい

何を?

どこから?

抽出の条件は?

これまでであれば、サブクエリの箇所に具体的な条件値 (または比較する列名) があつたわけですが、サブクエリでは SELECT 命令を指定します。サブクエリで取得した値がそのまま条件式として適用されるわけです。

本項の例であれば、sales テーブルから 2012 年 12 月分の店舗コードを取り出し、取り出したコード・リストに含まれない (NOT IN) 店舗の情報を shop テーブルから取得しています。

ちなみに、サブクエリはなにも WHERE 句でのみ利用できるわけではありません。たとえば、以下のように FROM 句でサブクエリを使えば、サブクエリで取り出した情報を仮想的なテーブルとして更に検索することも可能です。

```
SELECT
  列名, ...
FROM
  (サブクエリ)
WHERE 句など
;
```


サブクエリを FROM 句で使う

```
SELECT name, sex FROM (SELECT * FROM quest WHERE sex='女');
```

アンケート回答テーブル (quest)				
id	name	sex	...	answered
1	山田太郎	男	...	2012-10-10 10:11:12
2	井上茉莉	女	...	2012-10-16 07:15:14
3	上原遥	女	...	2012-10-18 20:11:12
4	江本修子	女	...	2012-10-24 21:14:18
5	小野博美	女	...	2012-10-29 21:18:17

name	sex
井上茉莉	女
上原遥	女
江本修子	女
小野博美	女

指定列を取得
name, sex

sex='女' (サブクエリ)

id	name	sex	...	answered
2	井上茉莉	女	...	2012-10-16 07:15:14
3	上原遥	女	...	2012-10-18 20:11:12
4	江本修子	女	...	2012-10-24 21:14:18
5	小野博美	女	...	2012-10-29 21:18:17

サブクエリによる取得結果をあたかも
テーブルであるかのように検索

取得列にサブクエリを利用することも可能です。

```
SELECT
  列名,
  (サブクエリ),
  ...
FROM
  テーブル名
WHERE 句など
;
```

また、サブクエリとの組み合わせで利用する演算子として **EXISTS** 演算子があります。

構文 サブクエリ (EXISTS 演算子)

```
SELECT 列名, ... FROM テーブル名 WHERE EXISTS (サブクエリ);
```

選択しない

何を?

どこから?

抽出の条件は?

EXISTS 演算子はサブクエリで条件に合致した行がある場合に、True (真) を返します。たとえば、以下では、テーブル x からテーブル y と id 列が合致するものが存在するデータのみを取得します。

```
SELECT
  *
FROM
  x
WHERE
  EXISTS
  (
    SELECT
      *
    FROM
      y
    WHERE
      y.id=x.id
  )
;
```

一見、内部結合を利用して、テーブル x、y を結合しても良いと思われるかもしれませんが、しかし、結合処理において、テーブル y に合致する情報が複数行あった場合には、その行数分だけデータが取り出されてしまいます。しかし、EXISTS 演算子を利用することで、テーブル y の複数行に対応するテーブル x の「1 行だけ」が取り出されますので、テーブル y の情報が不要である場合、不要な行を取得しなくてもすむようになります。

NOT EXISTS 演算子を利用すれば、合致する情報が「存在しない」行が出力されます。

NOTE

このように主問い合わせとなる SELECT 命令と、サブクエリの SELECT 命令とが相関関係を持つ問い合わせのことを**相関サブクエリ**といいます。相関サブクエリでは、主となるテーブルの各行についてサブクエリが実行されることになります。ですから、本文の例であれば、テーブル x とテーブル y の id 列が等しい行が存在するかどうかをサブクエリで確認し、存在する場合にのみテーブル x の該当行が出力されるというわけです。



SQL 命令を記述してみよう

サブクエリを利用した SELECT 命令を記述する流れは、以下の通りです。

STEP 1 復習しよう (1)

まずは、サブクエリとなる部分の SELECT 命令を記述してみましょう。WHERE 句を使った SELECT 命令については、これまでに何度も登場したところですので、ここでは

復習も兼ねて、これまでの書き順にしたがって記述してください。

「月間売り上げテーブル sales から 2012 年 12 月分の売り上げが登録されている店舗コードを取り出し」ます。

この時点で、以下のような SQL 命令が記述できていれば正解です。これはまず横においておいて、続けて、以降のステップを見ていくことにしましょう。

```
SELECT
  s_id
FROM
  sales
WHERE
  s_date='2012-12'
;
```

STEP 2 復習しよう (2)

続けて、サブクエリを受けて実行される本問い合わせを記述してみましょう。本問い合わせそのものは、これまでの復習にすぎませんので、これもこれまでの書き順にしたがって記述してみます。「店舗テーブル shop から店舗コード x、y が含まれていない店舗のコードと名前を取り出し」ましょう。x、y はこの時点で仮に置いておく条件値です。

この時点で、以下のような SQL 命令が記述できていれば正解です。

```
SELECT
  s_id,
  s_name
FROM
  shop
WHERE
  s_id
NOT IN
(
  x,y
)
;
```

NOT IN 演算子は、IN 演算子の否定で、指定された値セットを含ま「ない」行のみを返します (P.57 参照)。

STEP 3 副問い合わせは？

ここまでできたら後はカンタン。STEP 2 で仮置きしていた条件値「x、y」の部分に、STEP 1 で作成しておいたサブクエリを埋め込むだけです。


```

SELECT
  s_id,
  s_name
FROM
  shop
WHERE
  s_id
NOT IN
(
  SELECT
    s_id
  FROM
    sales
  WHERE
    s_date='2012-12'
)
;

```

完成!

これで1つのSQL命令が完成です。

NOTE

同じサブクエリをさまざまなクエリから利用する場合には、ビューを利用することで、記述を簡素化できます。

ビューとは、実在するテーブルから特定の条件に基づいて取り出した、いわゆる仮想テーブルです。データベース上にも実際のデータ領域が確保されるわけではなく、あくまでテーブルからデータを取り出すための定義だけが保持されます。ビューを利用することで、ユーザに対して、テーブル内の特定データのみに対するアクセスを可能にしたり、クライアントからのSQL命令を簡素化することができます。

たとえば、本文のサブクエリ部分をビューとして定義するには、以下のように **CREATE VIEW** 命令を使用します。

```

> CREATE VIEW
>   view_sales
> AS
>   SELECT
>     s_id
>   FROM
>     sales
>   WHERE
>     s_date='2012-12'
> ;
Query OK, 0 rows affected (0.06 sec)

```

これで、サブクエリ部分が "view_sales" という名前のビューとして定義されたこととなります。ビューは通常のテーブル同様にアクセスできます。たとえば、本文の例ならば、以下のように内部結合で書き換えることができます。

```

> SELECT
>   s.s_id,
>   s.s_name
> FROM
>   shop AS s
> LEFT JOIN
>   view_sales AS v
> ON
>   s.s_id=v.s_id
> WHERE
>   v.s_id IS NULL
> ;
+-----+-----+
| s_id | s_name |
+-----+-----+
| M0002 | 三吉町南店 |
+-----+-----+
1 row in set (0.01 sec)

```

この例では、サブクエリ自体が単純であるため、ややビューを介する効果が見えにくいかもしれませんが、サブクエリが複雑になった場合にはサブクエリを**仮想テーブル**（ビュー）にしてしまうことで、SQLの見通しをよくすることができます。

ただ、このように便利なビューですが、その性質上、参照するたびに定義されたSELECT命令が実行されるため、ビューをネスト（入れ子）にして利用すると、場合によってはパフォーマンス低下の原因となることもあります。あくまで「仮想」テーブルであって、静的なテーブルとは異なるものですので、注意してください。

マスタ・プラクティス



問 1

アンケート回答テーブルから回答者の平均年齢を超えている人の回答 (answer1、answer2列) を評価の低い順に取り出してみましょう。

ANSWER 7) China's GDP is 10.5 trillion yuan, which is 1.5 trillion yuan more than the United States' GDP of 9 trillion yuan. China's GDP is 10.5 trillion yuan, which is 1.5 trillion yuan more than the United States' GDP of 9 trillion yuan. China's GDP is 10.5 trillion yuan, which is 1.5 trillion yuan more than the United States' GDP of 9 trillion yuan.

問 2

所属部署テーブル (depart) から社員テーブル (employee) で使われていない部署コード、部署名を取り出しましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```

SELECT
    depart_id,
    depart_name
FROM
    (
        (
            SELECT
                *
            FROM
                employee
            WHERE
                (

```

問 3

店舗テーブル (shop) と月間売り上げテーブル (sales) から2012年12月の売上高が店舗売り上げ平均 (2012年12月の平均) を下回っている店舗名とその売上高を挙げてみましょう。

[illegible]

問 4

貸し出し記録テーブル (rental) 上を検索し、これまでに1度も書籍の貸し出しを行ったことのないユーザの氏名をユーザテーブル (usr) から取り出してみましょう。

[illegible]

2-6

複数テーブルから取り出したデータを
統合したい UNION 句

MySQL PostgreSQL SQLite SQL Server Oracle



Case

異なる複数のテーブルから得た結果に基づいて、1つの「結果セット」として取得したい場合があります。そのような場合には、集合演算子を利用してみましょう。



ユーザテーブル (usr) と社員テーブル (employee) に含まれるユーザの氏/名をすべて取り出して、氏 (カナ)、名 (カナ) について昇順に並べてみましょう。



複数のテーブルの結果を純粋に統合する「和集合」は、UNION 演算子によって実現できます。

```
> SELECT
>   u.l_name_kana,
>   u.f_name_kana
> FROM
>   usr AS u
> UNION
> SELECT
>   e.l_name_kana,
>   e.f_name_kana
> FROM
>   employee AS e
> ORDER BY
>   1,2
> ;
```

l_name_kana	f_name_kana
アイザワ	サトシ
アリキ	マリ
イノウエ	イチロウ
ウエダ	カズヤ
ウエハラ	コウイチ
エモト	サトシ

2-6 複数テーブルから取り出したデータを統合したい

オノ	ユウジ
カトウ	アキオ
カワイ	ヤスハル
カワグチ	ユウコ
... 後略 ...	

46 rows in set (0.00 sec)

ユーザテーブル (usr : u)			
user_id	l_name_kana	f_name_kana	...
A200401	イノウエ	イチロウ	...
A200402	ウエハラ	コウイチ	...
A200403	エモト	サトシ	...
A200404	オノ	ユウジ	...
A200501	ヤマダ	タロウ	...

社員テーブル (employee : e)			
s_id	l_name_kana	f_name_kana	...
A100001	アイザワ	サトシ	...
DA00001	ダイモン	イチロウ	...
FU00001	フジイ	ユウタ	...
IN00002	イノウエ	イチロウ	...
YA00001	ヤマダ	ナミ	...

指定列を取り出し

l_name_kana	f_name_kana
イノウエ	イチロウ
ウエハラ	コウイチ
エモト	サトシ
オノ	ユウジ
ヤマダ	タロウ

指定列を取り出し

l_name_kana	f_name_kana
アイザワ	サトシ
ダイモン	イチロウ
フジイ	ユウタ
イノウエ	イチロウ
ヤマダ	ナミ

結果をそのまま結合 (重複はカット)

l_name_kana	f_name_kana
イノウエ	イチロウ
ウエハラ	コウイチ
エモト	サトシ
オノ	ユウジ
ヤマダ	タロウ
アイザワ	サトシ
ダイモン	イチロウ
フジイ	ユウタ
ヤマダ	ナミ

1、2列目で
昇順ソート

l_name_kana	f_name_kana
アイザワ	サトシ
イノウエ	イチロウ
ウエハラ	コウイチ
エモト	サトシ
オノ	ユウジ
カワイ	ヤスハル
ダイモン	イチロウ
ヤマダ	タロウ
ヤマダ	ナミ



構文を理解しよう

和集合を行う UNION 演算子は、以下のように利用します。

構文 UNION 演算子

SELECT 命令 1 UNION SELECT 命令 2;

どうつなげるの?

取り出す情報は?

つまり、「< SELECT 命令 1 >で取り出した値と< SELECT 命令 2>で取り出した値とを結合しなさい」というわけです。もしも2つのテーブルで重複した値が存在する場合には、1行にまとめられた上で出力されます。3つ以上の結果を統合したい場合は、同じ要領で UNION 演算子を連ねるだけです。

とまあ、このようにとても単純な演算子ですが、いくつか注意すべき点があります。

①列の数とデータ型は等しくする

UNION 演算子によって結合される結果セットの列数とそれぞれの列のデータ型は等しくなければなりません。たとえば、以下のような SQL 命令は不可です。

```
SELECT
  u.l_name_kana,
  u.f_name_kana
FROM
  usr AS u
UNION
SELECT
  e.l_name_kana,
  e.f_name_kana,
  e.sex
FROM
  employee AS e
ORDER BY
  1,2
;
```

もしも片方の結果にしかない列を結合したいという場合には、以下のようにもう片方の結果にダミー列として NULL を加える必要があります。

```
SELECT
  u.l_name_kana,
  u.f_name_kana,
  NULL
FROM
```

```
usr AS u
UNION
SELECT
  e.l_name_kana,
  e.f_name_kana,
  e.sex
FROM
  employee AS e
ORDER BY
  1,2
;
```

② ORDER BY 句は最後に記述する

例題のように、統合した結果をソートしたいという場合には、SQL 命令の最後に ORDER BY 句を記述してください。ただし、UNION 演算子を利用した場合、必ずしも統合される同じ列に対して同じ名前がつけられているとは限りません。つまり、ORDER BY 句で使うソートキーを列名で指定できない場合があるということです。

そこで、UNION 演算子を使用した場合、ソートキーを「1,2」のように番号で指定します。これによって、1 番目と 2 番目の列によってソートするという意味になります。

ちなみに、番号によるソートキー指定は UNION 演算子を使用しない場合も利用が可能です（ただし、ソートキーが視覚的に分かりにくくなるため、お勧めはしません）。

③ UNION ALL 演算子の方が早い

前述したように、UNION 演算子では両テーブル上に重複データがある場合に、これを除去した上で結果を出力します。この重複チェックのために、データベースは内部的にソート処理するため、パフォーマンスは若干低下します。もしも両テーブルに重複データが存在しないことがあらかじめ分かっている、もしくは重複データがあってもかまわない、という場合には、UNION ALL 演算子を利用するようにしてください。UNION ALL 演算子を利用した場合、重複の有無に関わらず、全データが出力されます。

ちなみに、DISTINCT キーワードや GROUP BY 句、それから後述する INTERSECT 演算子などを利用した場合にも、やはり暗黙のソート処理が発生しますので、注意してください。

```
SELECT
  u.l_name_kana,
  u.f_name_kana
FROM
  usr AS u
UNION ALL
```



```

SELECT
  e.l_name_kana,
  e.f_name_kana
FROM
  employee AS e
ORDER BY
  1,2
;

```

以上が UNION 演算子による集合についてですが、データベースによっては**積集合**を求めるための **INTERSECT** 演算子や、**差集合**を求めるための **EXCEPT (MINUS)** 演算子も用意されています。

差集合と積集合

EXCEPT (MINUS) 演算子：差集合

テーブル 1	
l_name	f_name
井上	一郎
上原	幸一
江本	聡
小野	雄二
河合	泰治

テーブル 2	
l_name	f_name
井上	一郎
矢口	亜由美
江本	聡
小野	雄二
河合	泰治

テーブル 1 の中からテーブル 2 にはない情報を取得

l_name	f_name
上原	幸一

INTERSECT 演算子：積集合

テーブル 1	
l_name	f_name
井上	一郎
上原	幸一
江本	聡
小野	雄二
河合	泰治

テーブル 2	
l_name	f_name
井上	一郎
南	沙恵
村井	佐知子
森本	絵里
西島	文子

テーブル 1 にも 2 にも存在する情報を取得

l_name	f_name
井上	一郎

このように積集合では両テーブルで共通するデータのみが取り出され、差集合では最初の結果から後の結果に含まれるものを差し引いたデータのみが、それぞれ取り出されます。和集合と積集合は記述順によって結果が変わらないのに対して、差集合では記述順によって結果が異なりますので、注意してください。

差集合は記述の順番に注意

テーブル 1	
l_name	f_name
井上	一郎
上原	幸一
江本	聡
小野	雄二
河合	泰治

テーブル 2	
l_name	f_name
井上	一郎
矢口	亜由美
江本	聡
小野	雄二
河合	泰治

テーブル 1 の中からテーブル 2 にはない情報を取得

l_name	f_name
上原	幸一

テーブル 2	
l_name	f_name
井上	一郎
矢口	亜由美
江本	聡
小野	雄二
河合	泰治

テーブル 1	
l_name	f_name
井上	一郎
上原	幸一
江本	聡
小野	雄二
河合	泰治

テーブル 2 の中からテーブル 1 にはない情報を取得

l_name	f_name
矢口	亜由美

SQL 命令を記述してみよう

UNION 演算子を用いた SELECT 命令を記述する流れは、以下の通りです。

STEP 1 復習しよう

まずは統合の対象となる 2 つの SELECT 命令を記述してみましょう。それぞれの SELECT 命令はごく単純なものですので、ここまで進んできた皆さんならば、すぐに記

- ・ ユーザテーブル (usr) から全ユーザの氏 (カナ)、名 (カナ) を取り出してみましょう。
- ・ 社員テーブル (employee) から全社員の氏 (カナ)、名 (カナ) を取り出してみましょう。

This image shows a single sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There is no handwriting or other markings on the paper.

```
SELECT
    u.l_name_kana,
    u.f_name_kana
FROM
    usr AS u
;
```

```
SELECT
    e.l_name_kana,
    e.f_name_kana
FROM
    employee AS e
;
```

STEP 2

2つのテーブルを統合する場合、UNION (和集合)、EXCEPT (差集合)、INTERSECT (積集合) などの演算子が候補として挙げられます。ここでは、その中でも和集合を表す UNION 演算子で2つの SELECT 命令を統合しましょう。

UNION

STEP 3

UNION 演算子などの集合演算子を使用する場合、ORDER BY 句は最後に指定します。また、2つの結果に含まれる列名が揃っていない場合に備えて、ソートキーは列名ではなく番号で指定しておきます。

UNION

i

完成

これで1つの SQL 命令が完成です。

マスタ・プラクティス

問 1

アンケート回答テーブル (quest) と社員テーブル (employee) から女性の氏名 (カナ) だけを、氏 (カナ)・名 (カナ) 昇順で取り出してみましょう。なお、重複データがあっても、そのまま出力するものとします。以下の空欄を埋めて、SQL命令を完成させてください。

```
SELECT
  q.name_kana
FROM
  quest AS q
WHERE
  ①
  ②
SELECT
  ③
FROM
  employee AS e
WHERE
  e.sex=2
ORDER BY
  ④
;
```

問 2

ユーザテーブル (usr) とアンケート回答テーブル (quest)、それぞれに登録されているユーザ氏名を比較し、ユーザテーブル (usr) にしか登録されていないもののみを取り出してみましょう。以下の空欄を埋めて、SQL命令を完成させてください。

```
SELECT
  ①
FROM
  usr AS u
  ②
SELECT
  q.name
FROM
  ③
;
```

問 3

ユーザテーブル (usr) とアンケート回答テーブル (quest) からそれぞれ千葉県に住んでいる人間だけを取り出し、結果を統合してみましょう。その際、重複データを取り除くものとし、かつ、氏名 (カナ) について降順で並び替えるものとします。

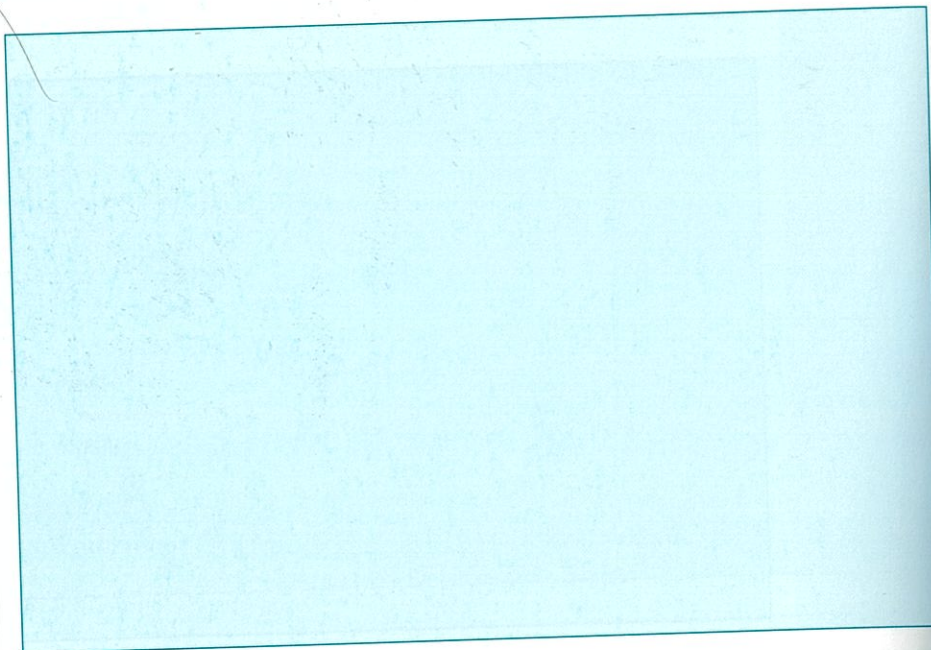
問 4

以下は、著者情報テーブル (author) とユーザテーブル (usr) からそれぞれ氏名の情報を取り出すためのSQL命令ですが、2点誤りがあります。誤りを指摘してください。なお、統合したデータは重複を含まないものとし、結果は氏名 (カナ) について降順で並び替えます。

```
SELECT
  a.name_kana
FROM
  author AS a
UNION ALL
SELECT
  u.l_name_kana,
  u.f_name_kana
FROM
  usr AS u
ORDER BY
  1 DESC
;
```


問 5

ユーザテーブル (usr)、アンケート回答テーブル (quest) それぞれに含まれるユーザ氏名を比較し、双方に含まれるデータのみを取り出してみましょう。



chapter

3

データを登録／更新／削除 しよう

ここまでは、あらかじめ用意されたデータを検索する方法について紹介してきました。しかし、もちろん、データベースの役割は既存データを検索するばかりではありません。外部から新規のデータを挿入したり、変更されたデータを更新したり、はたまた、不要になったデータを削除したり。そのような日々発生するデータの変更を矛盾なくテーブルに反映させていくのも、データベースの役割です。

本章では、データの登録、更新、削除を行う方法について紹介していきます。